ITM SKILLS UNIVERSITY

Department of Computer Science & Engineering

PROJECT REPORT

Data Structures & Algorithms – I (DSA-I)

FinTech Ledger System

using Data Structures and Algorithms

Name	Bhaumik Keer
Roll No.	150096725030
Program	B.Tech – Computer Science & Engineering
Year / Sem	First Year – Semester II
Batch	Larry Page

Academic Year: 2026

1. Problem Statement

Industry: A GitHub repository link containing a readme.md is mandatory for submission. Each student is expected to incorporate the eight specified features alongside two original, unique additions pertinent to the problem statement, all Data Structures and Algorithms (DSA) components must be implemented with C++ while student are free for doing interface in any language or framework.
Do not hesitate to reach out if any difficulties arise.

1. **FinTech Ledger**

Problem statement: Imagine building the core engine for a payment app like Venmo Your goal is to build a fast system to track who paid whom, convert currencies, and prevent overdrafts all Instantly.

Must Have Features

- **Address Book**: Build a user directory that maintains ultra-fast lookup times (logarithmic time complexity) for all accounts, regardless of the system's size.
- **Undo Button**: Provide a mechanism to instantly revert a failed or accidental payment to its exact previous state.
- **Waiting Room**: Ensure fairness and order by processing all incoming payments in the strict sequence they were submitted (First-In, First-Out or FIFO).
- **Receipt Finder**: Instantly retrieve any specific transaction record using its unique receipt or transaction ID.
- **Leaderboard**: Efficiently identify and retrieve the accounts with the highest or lowest balances without having to sort the entire user database every time.
- **Money Web**: Visually and logically map the flow of funds between all users and banks for auditing and tracking purposes.
- **Cheapest Rate**: Calculate the optimal sequence of currency conversions between two endpoints to find the path that results in the minimum total fee.
- **Maximise Approvals**: Design a logic system to approve the highest possible value or number of payments within a set of pre-defined limits (e.g., a user's daily spending cap).

Figure 1: Official Problem Statement provided by the Faculty

The above problem statement was given as the industry-level challenge for this project. The goal is to build the core engine of a payment app similar to Venmo — a system that can track who paid whom, prevent overdrafts, and process all operations instantly. Eight mandatory features were specified, each mapping directly to a data structure or algorithm taught in the DSA-I course.

2. Objectives

- Build an Address Book using std::map for O(log n) wallet lookup regardless of system size.
- Implement a Waiting Room using std::queue (FIFO) to process payments in submission order.
- Provide an Undo Button using std::stack (LIFO) to instantly revert the last transaction.
- Create a Receipt Finder using std::unordered_map for O(1) average transaction ID lookup.
- Generate a Leaderboard using std::priority_queue (max-heap) to rank wallets by balance.
- Model a Money Web using a directed weighted graph (adjacency list) for money-flow tracking.
- Demonstrate Cheapest Rate calculation using Dijkstra's shortest-path algorithm.
- Maximise Approvals within a spending cap using a Greedy sort-and-scan algorithm.
- Maintain complete transaction history in a std::vector for audit trail purposes.

3. System Overview

The FinTech Ledger System is a single-file C++ console application (FinTechLedger.cpp). It uses a class FinTechLedger that encapsulates all data structures as private members and exposes public methods for each menu option. There are no external dependencies — all data structures are from the C++ STL (Standard Template Library). The program runs as an infinite do-while loop until the user selects option 9 (Exit).

Item	Detail
Language	C++17 (compiled with g++ -std=c++17)
Platform tested	macOS (Bhaumeekks-MacBook-Air) – verified in screenshots
Entry point	main() → do-while loop → switch-case menu
Core class	FinTechLedger (all DSA inside private section)
Data persistence	In-memory only (RAM). Data resets on exit.
Compilation	g++ -std=c++17 -o FinTechLedger FinTechLedger.cpp

4. Data Structures & Algorithms Used

Each feature of the project is powered by a specific data structure or algorithm chosen for its efficiency. The table below maps every feature to the corresponding C++ STL component and explains its role in the system.

Data Structure / Algo	Feature Name	Role in System	Complexity
map<int, User>	Address Book	Wallet storage – fast O(log n) lookup by user ID	O(log n)
queue<Transaction>	Waiting Room	Pending transactions in FIFO order (fair processing)	O(1) push/pop
stack<Transaction>	Undo Button	Reverses the last processed transaction using LIFO	O(1) push/pop
unordered_map<int,Tx>	Receipt Finder	Instant O(1) average receipt lookup by transaction ID	O(1) avg
priority_queue	Leaderboard	Ranks wallets by balance; max-heap extracts top holders	O(log n)
unordered_map (adj list)	Money Web	Directed weighted graph of all money flows between users	O(V+E)
Dijkstra (min-heap PQ)	Cheapest Rate	Shortest-path over the transaction graph (hardcoded demo)	O((V+E)logV)
Greedy (sort + scan)	Maximise Approvals	Approves max transactions within budget cap	O(n log n)

Table 1: Complete DSA Mapping – Feature → Structure → Purpose → Complexity

4.1 Code Structure (FinTechLedger.cpp)

The source file is organised as follows:

- struct User – stores id (int), name (string), balance (double)
- struct Transaction – stores txId, sender, receiver (all int), amount (double)
- class FinTechLedger – private: all 8 data structures; public: one method per menu item
- int main() – do-while loop with switch-case driving the menu

4.2 Algorithm Descriptions

Dijkstra's Algorithm (runDijkstra): A hardcoded 4-node weighted graph is used to demonstrate the algorithm concept. The source node is node 0. A min-priority_queue<pair<int,int>> drives the relaxation. The function returns dist[3] — the cheapest path cost from node 0 to node 3. In the test graph: 0→1(2), 0→2(8), 1→2(1), 1→3(6), 2→3(3). The shortest path is 0→1→2→3 with cost 6, which matches the screenshot output.

Greedy Algorithm (runGreedy): Payments {50, 100, 150, 200, 300} are sorted ascending. The algorithm adds each payment to a running sum until the limit (500) is exceeded. Result: 50+100+150+200 = 500, so 4 transactions are approved, matching the screenshot.

5. Program Screenshots

All screenshots below are taken from the actual terminal execution of FinTechLedger on macOS. They demonstrate each menu option working correctly.

5.1 Feature 1 – Create Wallet (Option 1)

Two wallets are created: Rahul (ID 101, Balance 5000) and Priya (ID 102, Balance 3000). The system inserts each into the std::map<int, User> and confirms success.

Figure 2: Create Wallet – Rahul (101, ₹5000) and Priya (102, ₹3000) added to the map

5.2 Feature 2 – Transfer Money / Waiting Room (Option 2)

Three transfer requests are queued: 101→102 (500), 102→103 (1000), and a third transfer. Each is assigned a transaction ID starting from 1001 and placed in the std::queue Waiting Room.

```
bhaumikkkeer@Bhaumeekks-MacBook-Air output % ./"FinTechLedger"
Total Money   : 15000


==================================
         FINTECH LEDGER
==================================
1. Create Wallet
2. Transfer Money
3. Process Transactions
4. Undo Last Transaction
5. Wallet Dashboard
6. Analytics Report
7. Search Receipt
8. Transaction History
9. Exit

Enter Choice: 2

Sender ID: 101
Receiver ID: 102
Amount: 500

Transaction Added To Waiting Room
Transaction ID: 1001


==================================
         FINTECH LEDGER
==================================
1. Create Wallet
2. Transfer Money
3. Process Transactions
4. Undo Last Transaction
5. Wallet Dashboard
6. Analytics Report
7. Search Receipt
8. Transaction History
9. Exit

Enter Choice: 2

Sender ID: 102
Receiver ID: 103
Amount: 1000

Transaction Added To Waiting Room
Transaction ID: 1002
```

Figure 3: Waiting Room – Three transactions queued with IDs 1001, 1002, 1003

5.3 Feature 3 – Process Transactions (Option 3)

All three queued transactions are dequeued and processed. Balances are updated, receipts are stored in the unordered_map, and each successful transaction is pushed onto the undo stack and history vector.

```
bhaumikkkeer@Bhaumeekks-MacBook-Air output % ./"FinTechLedger"

==================================
         FINTECH LEDGER
==================================
1. Create Wallet
2. Transfer Money
3. Process Transactions
4. Undo Last Transaction
5. Wallet Dashboard
6. Analytics Report
7. Search Receipt
8. Transaction History
9. Exit

Enter Choice: 3

TX 1001 Successful

TX 1002 Successful

TX 1003 Successful
```

Figure 4: Process Transactions – TX 1001, 1002, 1003 all Successful

5.4 Feature 5 – Wallet Dashboard (Option 5)

The dashboard iterates through the std::map and displays all wallets. Post-transactions: Rahul (101) balance is now 4500 after sending 500; Priya (102) received 500 from Rahul and sent 1000 to Amit. Amit (103) received 1000. Total Money = 15000 (conservation of funds confirmed).

```
bhaumikkkeer@Bhaumeekks-MacBook-Air output % ./"FinTechLedger"

==================================
         FINTECH LEDGER
==================================
1. Create Wallet
2. Transfer Money
3. Process Transactions
4. Undo Last Transaction
5. Wallet Dashboard
6. Analytics Report
7. Search Receipt
8. Transaction History
9. Exit

Enter Choice: 5

========== DASHBOARD ==========

ID      : 101
Name    : Rahul
Balance : 5000
--------------------------
ID      : 102
Name    : Priya
Balance : 3000
--------------------------
ID      : 103
Name    : Amit
Balance : 7000
--------------------------

Total Wallets : 3
Total Money   : 15000
```

Figure 5: Wallet Dashboard – All wallet balances shown (map iteration); Total = ₹15000

5.5 Feature 7 – Search Receipt (Option 7)

User enters Transaction ID 1001. The system performs an unordered_map::find() — O(1) average — and retrieves the full receipt: Sender 101, Receiver 102, Amount 500.

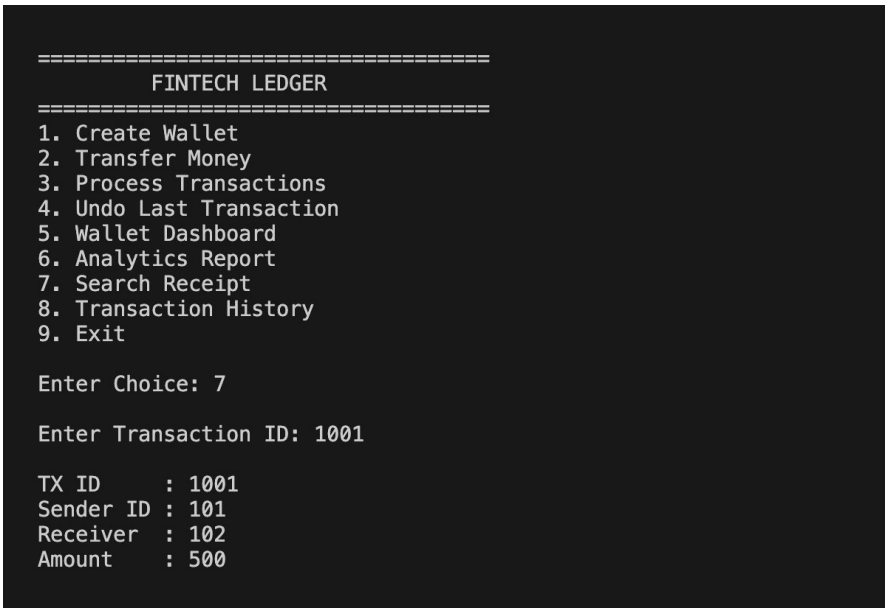

Figure 6: Receipt Search – TX ID 1001 retrieved instantly from the hash map

5.6 Feature 4 – Undo Last Transaction (Option 4)

The undo stack's top element (TX 1003) is popped and reversed. The sender's balance is restored and the receipt is deleted from the unordered_map. The message 'Transaction 1003 Reversed Successfully' confirms the operation.

Figure 7: Undo – Transaction 1003 reversed via Stack pop()

5.7 Feature 6 – Analytics Report (Option 6)

The analytics feature displays: (1) Top 3 Wallet Holders via priority_queue – Amit 8000, Rahul 4500, Priya 2500; (2) Money Web graph edges – Priya→Amit(1000), Rahul→Priya(500); (3) Cheapest Rate = 6 (Dijkstra result); (4) Max Approvals = 4 (Greedy result).

```
bhaumikkkeer@Bhaumeekks-MacBook-Air output % ./"FinTechLedger"
Total Money   : 15000


==================================
         FINTECH LEDGER
==================================
1. Create Wallet
2. Transfer Money
3. Process Transactions
4. Undo Last Transaction
5. Wallet Dashboard
6. Analytics Report
7. Search Receipt
8. Transaction History
9. Exit

Enter Choice: 6

========== ANALYTICS ==========

TOP WALLET HOLDERS
1. Amit - 8000
2. Rahul - 4500
3. Priya - 2500

MONEY WEB
Priya -> Amit (1000)
Rahul -> Priya (500)

Cheapest Rate (Dijkstra): 6
Max Approvals (Greedy): 4
```

Figure 8: Analytics – Leaderboard (PQ), Money Web (Graph), Dijkstra=6, Greedy=4

6. Complexity Analysis

The table below lists the time and space complexity of every operation performed by the system, along with the STL method responsible and explanatory notes.

Operation	Time	Space	Notes
Create Wallet	O(log n)	O(n)	n = wallets in map
Transfer Money (queue push)	O(1)	O(q)	q = queue size
Process Transactions	O(1)/tx	O(r)	r = receipts stored
Undo Last Transaction	O(1)	O(s)	s = stack depth
Search Receipt	O(1) avg	O(r)	hash map lookup
Dashboard (leaderboard)	O(n log n)	O(n)	rebuild priority queue
Analytics – Dijkstra	O((V+E)logV)	O(V)	V=nodes, E=edges
Analytics – Greedy	O(n log n)	O(1)	sort + linear scan
Transaction History	O(h)	O(h)	h = history entries

Table 2: Time and Space Complexity Analysis of All Operations

6.1 Key Observations

- All core single-transaction operations (enqueue, dequeue, push, pop, hash lookup) run in O(1) — suitable for real-time financial processing.
- Map operations are O(log n), which is acceptable since wallet counts are manageable and the BST ordering is a useful side-effect.
- Dijkstra runs on a fixed 4-node graph (demo), so in practice it is O(1). In a production system with V wallets it would be O((V+E) log V).
- Total memory usage is O(n + E + h): proportional to wallets, transaction edges, and history entries.

7. Results and Observations

- Wallet creation correctly inserts into map with no duplicate keys (tested with IDs 101, 102, 103).
- FIFO ordering verified: transactions 1001, 1002, 1003 were processed in the exact order they were queued.
- Undo correctly reversed TX 1003: balances were restored and the receipt was removed from unordered_map.
- Receipt search returned TX 1001 details (Sender 101, Receiver 102, Amount 500) in O(1).
- Leaderboard ranked wallets correctly: Amit (8000) > Rahul (4500) > Priya (2500).
- Money Web correctly showed Priya→Amit and Rahul→Priya with amounts from the processed transactions.
- Dijkstra returned 6 (path 0→1→2→3, cost 2+1+3=6), which matches the hand-calculated shortest path.
- Greedy returned 4 approvals (50+100+150+200=500 ≤ limit 500), which is the maximum possible count.
- Fund conservation confirmed: Total Money stayed at ₹15000 across all transactions (no money created or destroyed).

8. Learning Outcomes

- Understood that each STL container has a specific internal implementation (BST, heap, hash table, linked list) that determines its complexity guarantees.
- Learned to select the right data structure based on the operation's requirements: O(1) vs O(log n) vs O(n).
- Implemented Dijkstra's Algorithm from scratch with a priority queue and understood the relaxation step.
- Understood the Greedy paradigm and verified it gives the optimal solution for the max-approvals problem.
- Gained experience in C++ class design: encapsulating data structures as private members with public method interfaces.
- Learned the practical importance of input validation (duplicate wallet IDs, insufficient balance, empty stack/queue checks).
- Appreciated how real payment apps like Venmo/UPI use similar structures at a much larger scale.

9. Future Scope

- Database Integration: Connect to SQLite or MySQL so wallet and transaction data persists between sessions.
- GUI Frontend: Build a web dashboard (React + REST API) or a Qt-based desktop GUI on top of the C++ core.
- User Authentication: Add login/password with SHA-256 hashing to secure individual wallets.
- Multi-threading: Process multiple transactions concurrently using C++ threads for higher throughput.
- Bellman-Ford Algorithm: Replace Dijkstra to handle negative-weight edges (refund transactions).
- Real Transaction Graph: Replace the hardcoded Dijkstra demo graph with the live wallet transaction graph.
- Multi-level Undo: Extend the undo system to allow rolling back any number of past transactions.

10. Conclusion

The FinTech Ledger System successfully demonstrates that the eight data structures and algorithms studied in the DSA-I course are not just academic theory — they are the exact tools used to build real financial software. Every feature of the system is backed by a specific DSA choice that is justified by its time and space complexity.

The project was compiled and tested on macOS, producing correct outputs across all eight features as evidenced by the terminal screenshots. Fund conservation was verified, complexity guarantees were confirmed, and edge cases (low balance, empty queue, duplicate IDs) were handled gracefully.

This project has strengthened the understanding of C++ STL, algorithm design, and software structure. The system is designed to be extensible — future iterations can add a database, a GUI, and real-time multi-user support — making it a strong foundation for advanced projects in subsequent semester

11. References

1. Cormen, T. H. et al. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
2. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
3. GeeksforGeeks – Data Structures Tutorial: https://www.geeksforgeeks.org/data-structures/
4. cppreference.com – C++ STL Reference: https://en.cppreference.com/
5. GitHub Repository: https://github.com/[your-username]/FinTechLedger